

WEEK6_SPRINT1_SUMMARY

Sprint 1 - Cardiac Monitor API

What I set out to do

Finish the data model for the cardiac monitoring app, get it into real migrations, and then prove that the two flows that matter actually work: browsing patients, and placing a medication refill order that touches stock and money.

Where things landed

Everything I planned for the sprint is implemented and running. Below is what I built and where to find it.

The domain model lives in `Models/` and is wired up in `Data/AppDbContext.cs`. Patients, vitals, medications, appointments, and the new medication orders all have their relationships and delete rules defined explicitly rather than left to convention.

Seed data and migrations are in `Data/Migrations/`, with the seeding done in `AppDbContext.OnModelCreating`. A fresh database can be built from the migration chain alone.

The patient catalog (`GET /api/patients`) handles pagination, filtering by name and gender, sorting, and returns DTOs instead of EF entities. Both the happy path and the error paths have tests.

Medication order creation (`POST /api/patients/{patientId}/medication-orders`) checks that each medication exists, is active, and has enough stock, then calculates the line and order totals server-side, saves the order, and decrements stock. On a relational provider the whole thing runs inside a serializable transaction so two orders can't both take the last unit. Covered by service-level and API-level tests.

The demo is `Sprint1_Day5_Demo.postman_collection.json` - a collection I can run top to bottom in front of anyone without editing anything by hand.

Definition of Done

- Status codes are correct: `401` on the catalog without a token, `201 Created` on a successful order, `409 Conflict` when stock isn't enough.

- Every feature has both a success test and a failure test.
- No endpoint throws an unhandled exception - the global exception middleware has its own integration test.
- Two known security issues are still open (dev secrets in config, and unrestricted role selection at registration). Both are written down below so they don't get forgotten.

Running the demo

Import `Sprint1_Day5_Demo.postman_collection.json`, start the API with the HTTPS launch profile, and run the collection from the top. If Postman complains about the local dev certificate, turn off SSL verification - it's a local-only demo.

The collection walks through:

1. Log in as the seeded doctor and grab the JWT automatically.
2. Browse the patient catalog with pagination, two filters, and sorting.
3. Create an active medication with 5 units in stock and save its ID.
4. Show the stock before the order.
5. Place an order for 2 units and check the totals the server calculated.
6. Read the medication again - stock is now 3.
7. Try to order more than what's left and confirm it comes back `409 Conflict`.

Credentials are the dev seed ones (`doctor@cardiac.com` / `Doctor@123`). Obviously not production credentials.

ERD

Cardiac Monitor Chen-style ERD



Cardiac Monitor Chen-style ERD

This diagram covers the clinical and medication-order side of the app. I left the Identity role and refresh-token tables out on purpose - they're security plumbing, not part of the business domain.

Delete behavior, since it isn't obvious from the diagram alone: vitals, medications, and appointments cascade when a patient is deleted; a patient with medication orders can't be deleted at all; order items cascade from their order; and a medication that's referenced by an order item is protected from deletion.

Migrations so far

What it did	Migration
Identity tables plus patients, vitals, medications, appointments	<code>InitialCreate_20260814083124</code>
Roles, sample patients and vitals, nullable patient-user link	<code>SeedInitialData_20260814114402</code>
Dev doctor account and Doctor-role assignment	<code>AddSeedDoctor_20260814184132</code>

What it did	Migration
Auth seed state; the refresh-token schema itself ended up in a later migration	AddRefreshTokenTable_20260815174509
Stock and pricing, orders, order items, refresh tokens, constraints, indexes	AddMedicationOrders_20260828115119
Model-snapshot sync only, no SQL operations	SyncSprintOneModel_20260828121151

To apply and check:

```
dotnet ef database update
dotnet ef migrations list
```

What carries into Sprint 2

Move the JWT signature key outside the range followed; its source must be local user secrets, and environment variations or secret manager in the production environment.

Lock down role selection at registration. Right now a caller can ask for any existing role, including privileged ones. Public registration should always create a low-privilege user, and anything above that should be admin-only.

Run the Postman collection against real SQL Server. The integration tests use EF Core InMemory, which can't prove that the serializable transaction behaves the way I expect. I want to see the stock decrement in SQL Server with my own eyes.

Add a concurrency test on a real database for the last-unit case: two competing orders, exactly one succeeds, stock never goes negative.

Tidy up the migration history. The refresh-token migration name doesn't match where the table actually got created. Not urgent - a clean database still builds correctly from the chain - and I'd only squash before a shared production release.

Retrospective

What went well

Controllers stayed thin; the catalog and order rules live in services where they belong. Read queries project straight to response DTOs, so no EF entities leak

out of the API. The order flow keeps validation, totals, persistence, and stock changes in one coherent place instead of scattering them. And by the end of the sprint both the success and failure paths were proven at the HTTP boundary, not just in unit tests.

What didn't

I left the demo and the write-up until Day 5 instead of keeping them current as I went, which made the last day heavier than it needed to be. The original Postman collection had half-finished requests in it, so it was useless as a review script and I had to rebuild it. The two security issues above are debt I created and then had to write down rather than fix. And the InMemory test setup gave me a false sense of coverage on the transaction work.

One thing I'm changing

For every Sprint 2 endpoint, I'll write the Postman example and both acceptance tests (happy path and error path) *before* the implementation, and a card doesn't move to Done until all three pass.

For the check-in

What I'll have ready to show:

- This summary and the ERD.
- A green `dotnet test .\CardiacMonitor.slnx --configuration Release` run.
- The Postman Collection Runner with all seven requests passing.
- The medication row in SQL Server before and after the order goes through.